GURU GHASIDAS CENTRAL UNIVERSITY

BILASPUR

LAB REPORT

Object Oriented Programming with Java

Submitted by :- Tanushree Priyadarshini Sahoo

Guided By :- Dr. Ratnesh Srivastava

Enrollment Number :- GGV/24/05172

Roll Number :- 24070172

Course :- Bsc. C.S. (4th semester)

PROGRAM 1

//Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name

```
import java.io.*;

import java.util.*;


public class maincode {


  static class Farmer {

    private String name, city, fruit;

    private int quantity;


    public Farmer(String name, String city, String fruit, int quantity) {

      this.name = name;

      this.city = city;

      this.fruit = fruit;

      this.quantity = quantity;

    }


    public String getName() { return name; }

    public String getCity() { return city; }

    public String getFruit() { return fruit; }

    public int getQuantity() { return quantity; }

  }


  //  Display table

  public static void displayTable(List<Farmer> farmers) {

    System.out.printf("%-15s %-15s %-15s %-10s\n",

          "Name", "City", "Fruit", "Quantity");

    System.out.println("-----------------------------------------------------------");
```

```
    for (Farmer f : farmers) {
        System.out.printf("%-15s %-15s %-15s %-10d\n",
                f.getName(), f.getCity(), f.getFruit(), f.getQuantity());
    }
}

// Write to file
public static void writeToFile(List<Farmer> farmers, String filename) {
    try {
        BufferedWriter writer = new BufferedWriter(new FileWriter(filename));

        writer.write(String.format("%-15s %-15s %-15s %-10s\n",
                "Name", "City", "Fruit", "Quantity"));
        writer.write("-----------------------------------------------------------\n");

        for (Farmer f : farmers) {
            writer.write(String.format("%-15s %-15s %-15s %-10d\n",
                    f.getName(), f.getCity(), f.getFruit(), f.getQuantity()));
        }

        writer.close();
    } catch (IOException e) {
        System.out.println("File error");
    }
}
// SEARCH FUNCTION
public static void searchFarmer(List<Farmer> farmers, String name) {
    boolean found = false;
    System.out.println("\n--- SEARCH RESULT ---");
    System.out.printf("%-15s %-15s %-15s %-10s\n",
```

```
            "Name", "City", "Fruit", "Quantity");
        System.out.println("-----------------------------------------------------------");

        for (Farmer f : farmers) {
            if (f.getName().equalsIgnoreCase(name)) {
                System.out.printf("%-15s %-15s %-15s %-10d\n",
                        f.getName(), f.getCity(), f.getFruit(), f.getQuantity());
                found = true;
            }
        }

        if (!found) {
            System.out.println("Farmer not found");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        //  Input
        for (int i = 0; i < 10; i++) {
            System.out.println("\nEnter details for Farmer " + (i + 1));

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("City: ");
            String city = sc.nextLine();

            System.out.print("Fruit: ");
```

```
        String fruit = sc.nextLine();

        System.out.print("Quantity: ");
        int quantity = sc.nextInt();
        sc.nextLine();

        farmers.add(new Farmer(name, city, fruit, quantity));
    }

    //  BEFORE SORTING
    System.out.println("\n--- BEFORE SORTING ---");
    displayTable(farmers);
    writeToFile(farmers, "before_sort.txt");

    //  SORT BY QUANTITY
    farmers.sort((f1, f2) -> f1.getQuantity() - f2.getQuantity());

    //  AFTER SORTING
    System.out.println("\n--- AFTER SORTING ---");
    displayTable(farmers);
    writeToFile(farmers, "after_sort.txt");

    //  SEARCH
    System.out.print("\nEnter farmer name to search: ");
    String searchName = sc.nextLine();

    searchFarmer(farmers, searchName);

    sc.close();
  }
}
```

OUTPUT-

```
> cd "d:\" ; if ($?) { javac maincode.java } ; if ($?) { java maincode }

Enter details for Farmer 1
Name: ABCD
City: BHUBANESWAR
Fruit: APPLE
Quantity: 500

Enter details for Farmer 2
Name: BCDE
City: RAIGARH
Fruit: MANGO
Quantity: 300

Enter details for Farmer 3
Name: EFGH
City: CUTTACK
Fruit: LITCHI
Quantity: 670

Enter details for Farmer 4
Name: IJKL
City: BILASPUR
Fruit: ORANGE
Quantity: 500

Enter details for Farmer 5
Name: MNOP
City: INDORE
Fruit: BANANA
Quantity: 30

Enter details for Farmer 6
Name: QRST
City: HYDERABAD
Fruit: BLUEBERRY
Quantity: 20
```

```
Enter details for Farmer 6
Name: QRST
City: HYDERABAD
Fruit: BLUEBERRY
Quantity: 20

Enter details for Farmer 7
Name: ILJH
City: DELHI
Fruit: GUAVA
Quantity: 50

Enter details for Farmer 8
Name: VHGU
City: CHENNAI
Fruit: STRAWBERRY
Quantity: 400

Enter details for Farmer 9
Name: AFGH
City: KOLKATA
Fruit: COCONUT
Quantity: 400

Enter details for Farmer 10
Name: HKIT
City: JODHPUR
Fruit: RASBERRY
Quantity: 600
```

```
--- BEFORE SORTING ---
Name            City            Fruit           Quantity
-------------------------------------------------------
ABCD            BHUBANESWAR     APPLE           500
BCDE            RAIGARH         MANGO           300
EFGH            CUTTACK         LITCHI          670
IJKL            BILASPUR        ORANGE          500
MNOP            INDORE          BANANA          30
QRST            HYDERABAD       BLUEBERRY       20
ILJH            DELHI           GUAVA           50
VHGU            CHENNAI         STRAWBERRY      400
AFGH            KOLKATA         COCONUT         400
HKIT            JODHPUR         RASBERRY        600

--- AFTER SORTING ---
Name            City            Fruit           Quantity
-------------------------------------------------------
QRST            HYDERABAD       BLUEBERRY       20
MNOP            INDORE          BANANA          30
ILJH            DELHI           GUAVA           50
BCDE            RAIGARH         MANGO           300
VHGU            CHENNAI         STRAWBERRY      400
AFGH            KOLKATA         COCONUT         400
ABCD            BHUBANESWAR     APPLE           500
IJKL            BILASPUR        ORANGE          500
HKIT            JODHPUR         RASBERRY        600
EFGH            CUTTACK         LITCHI          670

Enter farmer name to search: MNOP

--- SEARCH RESULT ---
Name            City            Fruit           Quantity
-------------------------------------------------------
MNOP            INDORE          BANANA          30
```

PROGRAM 2-

//Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```
import java.util.*;


class Fruit {
  String name;
  int quantity;
  String quality;

  // Constructor
  Fruit(String name, int quantity, String quality) {
    this.name = name;

    // If quantity > 1000, store half
    if (quantity > 1000) {
      this.quantity = quantity / 2;
    } else {
      this.quantity = quantity;
    }

    this.quality = quality.toLowerCase();
  }

  // Method to display fruit details
  void display() {
    System.out.println("Fruit: " + name +
        ", Quantity: " + quantity +
        ", Quality: " + quality);
  }
}
```

```
public class FruitSearch {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // List to store fruits
    List<Fruit> fruits = new ArrayList<>();

    String[] fruitNames = {"Mango", "Apple", "Papaya"};

    // Input for fruits
    for (String fname : fruitNames) {
      System.out.println("Enter quantity for " + fname + ": ");
      int qty = sc.nextInt();

      System.out.println("Enter quality (good/bad/average) for " + fname + ": ");
      String quality = sc.next();

      fruits.add(new Fruit(fname, qty, quality));
    }

    // Display stored data
    System.out.println("\nStored Fruit Data:");
    for (Fruit f : fruits) {
      f.display();
    }

    // Search
    System.out.println("\nEnter fruit name to search: ");
    String searchName = sc.next();
```

```
        System.out.println("Enter quality to search: ");
        String searchQuality = sc.next().toLowerCase();

        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(searchName) &&
                f.quality.equals(searchQuality)) {

                System.out.println("\nFruit Found:");
                f.display();
                found = true;
            }
        }

        if (!found) {
            System.out.println("\nFruit with given quality not found.");
        }

        sc.close();
    }
}
```

OUTPUT-

```
> cd "d:\" ; if ($?) { javac FruitSearch.java } ; if ($?) { java FruitSearch }
Enter quantity for Mango:
1000
Enter quality (good/bad/average) for Mango:
good
Enter quantity for Apple:
400
Enter quality (good/bad/average) for Apple:
average
Enter quantity for Papaya:
1500
Enter quality (good/bad/average) for Papaya:
bad

Stored Fruit Data:
Fruit: Mango, Quantity: 1000, Quality: good
Fruit: Apple, Quantity: 400, Quality: average
Fruit: Papaya, Quantity: 750, Quality: bad

Enter fruit name to search:
mango
Enter quality to search:
good

Fruit Found:
Fruit: Mango, Quantity: 1000, Quality: good
```

PROGRAM 3-

// There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.The available orchards in a tree-view.Search for a given fruit in the available orchards after taking input from the user & print the tree.

```
import java.util.*;


// Node class for tree
class Node {
  String orchardName;
  String fruit;
  String quality;
  List<Node> children;


  Node(String orchardName, String fruit, String quality) {
    this.orchardName = orchardName;
    this.fruit = fruit;
    this.quality = quality;
    this.children = new ArrayList<>();
  }


  void addChild(Node child) {
    children.add(child);
  }
}


public class OrchardTree {


  // Function to print tree
  static void printTree(Node node, String indent) {
    if (node == null) return;
```

```
    System.out.println(indent + node.orchardName +
        " -> Fruit: " + node.fruit +
        ", Quality: " + node.quality);

    for (Node child : node.children) {
        printTree(child, indent + "   ");
    }
}

// Function to search fruit
static void searchFruit(Node node, String fruit) {
    if (node == null) return;

    if (node.fruit.equalsIgnoreCase(fruit)) {
        System.out.println("Found in Orchard: " + node.orchardName +
                           " (Quality: " + node.quality + ")");
    }

    for (Node child : node.children) {
        searchFruit(child, fruit);
    }
}

public static void main(String[] args) {

    // Root node
    Node root = new Node("Orchards", "", "");

    // Creating orchards
    Node A = new Node("A", "Apple", "Good");
    Node B = new Node("B", "Litchi", "Good");
```

```
        Node C = new Node("C", "Mango", "Good");
        Node D = new Node("D", "Apple", "Bad");

        // Building tree
        root.addChild(A);
        root.addChild(B);
        root.addChild(C);
        root.addChild(D);

        // Print tree
        System.out.println("Tree Structure:");
        printTree(root, "");

        // User input
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit to search: ");
        String searchFruit = sc.nextLine();

        // Search
        System.out.println("\nSearch Results:");
        searchFruit(root, searchFruit);

        sc.close();
    }
}
```

OUTPUT-

```
PS D:\> cd "d:\" ; if ($?) { javac OrchardTree.java } ; if ($?) { java OrchardTree }
Tree Structure:
Orchards -> Fruit: , Quality:
   A -> Fruit: Apple, Quality: Good
   B -> Fruit: Litchi, Quality: Good
   C -> Fruit: Mango, Quality: Good
   D -> Fruit: Apple, Quality: Bad

Enter fruit to search: APPLE

Search Results:
Found in Orchard: A (Quality: Good)
Found in Orchard: D (Quality: Bad)
```

PROGRAM 4-

//Take a map provided as a graph & display all the subtrees of connected points.

```
import java.util.*;


import java.util.ArrayList;

import java.util.Arrays;

import java.util.HashMap;

import java.util.HashSet;

import java.util.List;

import java.util.Map;

import java.util.Set;


public class GraphSubtrees {


  // Function to perform DFS

  static void dfs(int node, Map<Integer, List<Integer>> graph,

            Set<Integer> visited, List<Integer> component) {


    visited.add(node);

    component.add(node);


    for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {

      if (!visited.contains(neighbor)) {

        dfs(neighbor, graph, visited, component);

      }

    }

  }


  public static void main(String[] args) {


    // Graph representation (map)
```

```
        Map<Integer, List<Integer>> graph = new HashMap<>();

        // Example graph
        // Component 1: 1-2-3
        // Component 2: 4-5
        // Component 3: 6 (isolated)

        graph.put(1, Arrays.asList(2));
        graph.put(2, Arrays.asList(1, 3));
        graph.put(3, Arrays.asList(2));

        graph.put(4, Arrays.asList(5));
        graph.put(5, Arrays.asList(4));

        graph.put(6, new ArrayList<>()); // isolated node

        Set<Integer> visited = new HashSet<>();

        System.out.println("Subtrees (Connected Components):");

        for (int node : graph.keySet()) {
            if (!visited.contains(node)) {
                List<Integer> component = new ArrayList<>();

                dfs(node, graph, visited, component);

                System.out.println(component);
            }
        }
    }
}
```

OUTPUT-

```
> cd "d:\" ; if ($?) { javac GraphSubtrees.java } ; if ($?) { java GraphSubtrees }
Subtrees (Connected Components):
[1, 2, 3]
[4, 5]
[6]
```

PROGRAM 5-

//Enter entry point & goal point & produce the matching subtree of the graph.

```
import java.util.*;


import java.util.ArrayList;

import java.util.Arrays;

import java.util.HashMap;

import java.util.HashSet;

import java.util.List;

import java.util.Map;

import java.util.Scanner;

import java.util.Set;


public class GraphPath {


  // DFS to find path

  static boolean dfs(int current, int goal,

              Map<Integer, List<Integer>> graph,

              Set<Integer> visited,

              List<Integer> path) {


    visited.add(current);

    path.add(current);


    if (current == goal) {

      return true;

    }


    for (int neighbor : graph.getOrDefault(current, new ArrayList<>())) {

      if (!visited.contains(neighbor)) {

        if (dfs(neighbor, goal, graph, visited, path)) {
```

```
            return true;
        }
    }
  }

  // backtrack
  path.remove(path.size() - 1);
  return false;
}

public static void main(String[] args) {

  // Graph (example map)
  Map<Integer, List<Integer>> graph = new HashMap<>();

  graph.put(1, Arrays.asList(2, 3));
  graph.put(2, Arrays.asList(1, 4));
  graph.put(3, Arrays.asList(1, 5));
  graph.put(4, Arrays.asList(2));
  graph.put(5, Arrays.asList(3, 6));
  graph.put(6, Arrays.asList(5));

  Scanner sc = new Scanner(System.in);

  System.out.print("Enter entry point: ");
  int start = sc.nextInt();

  System.out.print("Enter goal point: ");
  int goal = sc.nextInt();

  List<Integer> path = new ArrayList<>();
```

```
        Set<Integer> visited = new HashSet<>();


        if (dfs(start, goal, graph, visited, path)) {
            System.out.println("Matching Subtree (Path): " + path);
        } else {
            System.out.println("No path found between " + start + " and " + goal);
        }


        sc.close();
    }
}
```

OUTPUT-

```
PS D:\> cd "d:\" ; if ($?) { javac GraphPath.java } ; if ($?) { java GraphPath }
Enter entry point: 1
Enter goal point: 6
Matching Subtree (Path): [1, 3, 5, 6]
```

PROGRAM 6-

//Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.*;


class Pair {
   int node, weight;


   Pair(int node, int weight) {
      this.node = node;
      this.weight = weight;
   }
}


public class DijkstraShortestPath {


   public static void main(String[] args) {


      // Graph with weights
      Map<Integer, List<Pair>> graph = new HashMap<>();


      // Example graph
      graph.put(1, Arrays.asList(new Pair(2, 4), new Pair(3, 2)));
      graph.put(2, Arrays.asList(new Pair(1, 4), new Pair(4, 5)));
      graph.put(3, Arrays.asList(new Pair(1, 2), new Pair(4, 8), new Pair(5, 10)));
      graph.put(4, Arrays.asList(new Pair(2, 5), new Pair(3, 8), new Pair(6, 6)));
      graph.put(5, Arrays.asList(new Pair(3, 10), new Pair(6, 3)));
      graph.put(6, Arrays.asList(new Pair(4, 6), new Pair(5, 3)));


      Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter start node: ");
int start = sc.nextInt();

System.out.print("Enter goal node: ");
int goal = sc.nextInt();

// Distance map
Map<Integer, Integer> dist = new HashMap<>();
Map<Integer, Integer> parent = new HashMap<>();

for (int node : graph.keySet()) {
    dist.put(node, Integer.MAX_VALUE);
}

dist.put(start, 0);

// Min-heap
PriorityQueue<Pair> pq = new PriorityQueue<>(Comparator.comparingInt(p -> p.weight));
pq.add(new Pair(start, 0));

while (!pq.isEmpty()) {
    Pair current = pq.poll();
    int node = current.node;

    for (Pair neighbor : graph.getOrDefault(node, new ArrayList<>())) {
        int newDist = dist.get(node) + neighbor.weight;

        if (newDist < dist.get(neighbor.node)) {
            dist.put(neighbor.node, newDist);
            parent.put(neighbor.node, node);
            pq.add(new Pair(neighbor.node, newDist));
```

```
            }
        }
    }

    // Reconstruct path
    List<Integer> path = new ArrayList<>();
    int current = goal;

    while (current != start) {
        path.add(current);
        current = parent.getOrDefault(current, -1);

        if (current == -1) {
            System.out.println("No path exists!");
            return;
        }
    }

    path.add(start);
    Collections.reverse(path);

    // Output
    System.out.println("Shortest Path: " + path);
    System.out.println("Total Cost: " + dist.get(goal));

    sc.close();
  }
}
```

OUTPUT-

```
PS D:\> cd "d:\" ; if ($?) { javac DijkstraShortestPath.java } ; if ($?) { java DijkstraShortestPath }
Enter start node: 5
Enter goal node: 12
No path exists!
PS D:\> cd "d:\" ; if ($?) { javac DijkstraShortestPath.java } ; if ($?) { java DijkstraShortestPath }
Enter start node: 1
Enter goal node: 6
Shortest Path: [1, 2, 4, 6]
PS D:\>
```

PROGRAM 7-

//Write a program to demonstrate the use of all keywords made for exception handling.

```
import java.util.*;


import java.util.Scanner;


public class ExceptionDemo {


  // Method using 'throws'
  static void checkAge(int age) throws Exception {
    if (age < 18) {
      // using 'throw'
      throw new Exception("Not eligible to vote");
    } else {
      System.out.println("Eligible to vote");
    }
  }


  public static void main(String[] args) {


    Scanner sc = new Scanner(System.in);


    try {
      // Input
      System.out.print("Enter your age: ");
      int age = sc.nextInt();


      // Calling method that uses throws
      checkAge(age);


      // Example of runtime exception
```

```
        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        int result = 100 / num;  // may cause ArithmeticException
        System.out.println("Result: " + result);

    }
    catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero!");
    }
    catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
    finally {
        // always executes
        System.out.println("Execution completed (finally block)");
    }

    sc.close();
  }
}
```

OUTPUT-

```
      > cd "d:\" ; if ($?) { javac ExceptionDemo.java } ; if ($?) { java ExceptionDemo }
Enter your age: 13
Exception: Not eligible to vote
Execution completed (finally block)
PS D:\> cd "d:\" ; if ($?) { javac ExceptionDemo.java } ; if ($?) { java ExceptionDemo }
Enter your age: 21
Eligible to vote
Enter a number: 0
Cannot divide by zero!
Execution completed (finally block)
PS D:\>
```

PROGRAM 8-

//Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException

```
import java.util.*;


// Custom Exception
class NoMoneyException extends Exception {
   public NoMoneyException(String message) {
      super(message);
   }
}


// Bank Account class
class BankAccount {
   double balance;


   BankAccount(double balance) {
      this.balance = balance;
   }


   // Method to withdraw money
   void withdraw(double amount) throws NoMoneyException {
      if (amount > balance) {
         throw new NoMoneyException("Insufficient balance! Withdrawal failed.");
      } else {
         balance -= amount;
         System.out.println("Withdrawal successful!");
         System.out.println("Remaining balance: " + balance);
      }
   }
}
```

```java
public class BankDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        double balance = sc.nextDouble();

        BankAccount account = new BankAccount(balance);

        try {
            System.out.print("Enter amount to withdraw: ");
            double amount = sc.nextDouble();

            account.withdraw(amount);

        } catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("Transaction completed.");
        }

        sc.close();
    }
}
```

OUTPUT-

```
PS D:\> cd "d:\" ; if ($?) { javac BankDemo.java } ; if ($?) { java BankDemo }
Enter initial balance: 2000
Enter amount to withdraw: 2500
Exception: Insufficient balance! Withdrawal failed.
Transaction completed.
PS D:\> cd "d:\" ; if ($?) { javac BankDemo.java } ; if ($?) { java BankDemo }
Enter initial balance: 2000
Enter amount to withdraw: 1000
Withdrawal successful!
Remaining balance: 1000.0
Transaction completed.
```

PROGRAM 9-

//Write a program which enables to use a platform by a train one at a time.

```
class Platform {

  private boolean isOccupied = false;

  // Method for train to use platform
  public synchronized void usePlatform(String trainName) {

    // If platform is busy, wait
    while (isOccupied) {
      try {
        System.out.println(trainName + " is waiting for the platform...");
        wait();
      } catch (InterruptedException e) {
        System.out.println(e);
      }
    }

    // Occupy platform
    isOccupied = true;
    System.out.println(trainName + " is entering the platform...");

    try {
      Thread.sleep(2000); // simulate usage time
    } catch (InterruptedException e) {
      System.out.println(e);
    }

    System.out.println(trainName + " is leaving the platform...");
```

```
        // Free platform
        isOccupied = false;

        // Notify other waiting trains
        notify();
    }
}

// Train thread
class Train extends Thread {
    Platform platform;
    String name;

    Train(Platform p, String name) {
        this.platform = p;
        this.name = name;
    }

    public void run() {
        platform.usePlatform(name);
    }
}

public class RailwayStation {
    public static void main(String[] args) {

        Platform platform = new Platform();

        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");
```

```
        t1.start();

        t2.start();

        t3.start();

    }

}
```

OUTPUT-

```
PS D:\> cd "d:\" ; if ($?) { javac RailwayStation.java } ; if ($?) { java RailwayStation }
Train A is entering the platform...
Train A is leaving the platform...
Train B is entering the platform...
Train B is leaving the platform...
Train C is entering the platform...
Train C is leaving the platform...
```

PROGRAM 10-

// Write a programme of Restaurant and Waiter, which accept order if kitchen if free and chef deliver orders if he is free.

```
class Kitchen {


  private String order;

  private boolean isOrderReady = false;


  // Waiter places order

  public synchronized void placeOrder(String order) {

    while (isOrderReady) {

      try {

        System.out.println("Waiter waiting, kitchen is busy...");

        wait();

      } catch (InterruptedException e) {

        System.out.println(e);

      }

    }


    this.order = order;

    isOrderReady = true;

    System.out.println("Waiter placed order: " + order);


    notify();

  }


  // Chef prepares order

  public synchronized void prepareOrder() {

    while (!isOrderReady) {

      try {

        System.out.println("Chef waiting, no orders...");
```

```
            wait();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }

    System.out.println("Chef is preparing: " + order);

    try {
        Thread.sleep(2000); // simulate cooking time
    } catch (InterruptedException e) {
        System.out.println(e);
    }

    System.out.println("Chef delivered: " + order);

    isOrderReady = false;

    notify();
  }
}

// Waiter Thread
class Waiter extends Thread {
  Kitchen kitchen;

  Waiter(Kitchen k) {
    this.kitchen = k;
  }

  public void run() {
```

```
        String[] orders = {"Pizza", "Burger", "Pasta"};

        for (String o : orders) {
            kitchen.placeOrder(o);

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Kitchen kitchen;

    Chef(Kitchen k) {
        this.kitchen = k;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            kitchen.prepareOrder();
        }
    }
}

// Main class
public class Restaurant {
```

```
    public static void main(String[] args) {

        Kitchen kitchen = new Kitchen();

        Waiter waiter = new Waiter(kitchen);
        Chef chef = new Chef(kitchen);

        waiter.start();
        chef.start();
    }
}
```

OUTPUT-

```
> cd "d:\" ; if ($?) { javac Restaurant.java } ; if ($?) { java Restaurant }
Waiter placed order: Pizza
Chef is preparing: Pizza
Chef delivered: Pizza
Chef waiting, no orders...
Waiter placed order: Burger
Chef is preparing: Burger
Chef delivered: Burger
Chef waiting, no orders...
Waiter placed order: Pasta
Chef is preparing: Pasta
Chef delivered: Pasta
```